LAPORAN

UJIAN TEGAH SEMESTER

4143108 – SOFTWARE ARCHITECTURE AND DESIGN

Disusun Oleh :

NIM	NAMA
11423018	Eduward Gilbert Simanjuntak

SARJANA TERAPAN TEKNOLOGI REKAYASA PERANGKAT LUNAK (TRPL)

INSTITUT TEKNOLOGI DEL

2026

Daftar ISI

Bab 1. ISI 3
2. 1 Case Study 1: MediTrack A Digital Healthcare Platform 3
2.1.1. Design Thinking & Architecture Selection 3
2.1.2. System Decomposition & Modeling 5
2.1.3. Architecture Visualization 9
2.1.4. Implementasi (Source Code Overview) 10
2.1.5. Bonus Task: Performance Bottleneck & Mitigation 11
2.2 Case Study 2: MediTrack Transformation 12
2.2.1. Use Case Based On The Case Study 12
2.2.2. Architecture Diagram 14
2.2.3. Reasoning and Trade-offs 14
2.2.4. Migration Strategy 14
2.2.5. Implementasi (Monolith Representastion) 14
2.2.6. Microservices Mapping 15

Bab 1. ISI

2. 1 Case Study 1: MediTrack A Digital Healthcare Platform

2.1.1. Design Thinking & Architecture Selection

Dalam pengembangan platform MediTrack, dipilih pendekatan Monolithic Architecture dengan konsep Modular Monolith menggunakan framework Laravel (PHP). Pendekatan ini dipilih untuk menyeimbangkan kebutuhan pengembangan cepat, kemudahan pemeliharaan, serta kesiapan terhadap pertumbuhan sistem di masa depan.

a. Justify the choice with at least three architectural principles

Pemilihan arsitektur Modular Monolith pada MediTrack didasarkan pada penerapan beberapa prinsip arsitektur perangkat lunak yang mendukung kualitas sistem secara keseluruhan. Salah satu prinsip utama yang digunakan adalah *Separation of Concerns*, di mana setiap bagian sistem dipisahkan berdasarkan tanggung jawabnya masing-masing. Dalam konteks ini, setiap modul seperti User Management, Appointment, EHR, Pharmacy, Payment, dan Analytics dikembangkan secara terpisah secara logis sehingga tidak saling mencampuri fungsi satu sama lain. Hal ini bertujuan untuk mengurangi kompleksitas sistem serta mempermudah proses pengembangan dan pemeliharaan.

Selain itu, prinsip *High Cohesion and Low Coupling* juga diterapkan untuk memastikan bahwa setiap modul memiliki fokus yang jelas terhadap fungsinya, sekaligus meminimalkan ketergantungan antar modul. Dengan cara ini, perubahan yang terjadi pada satu bagian sistem tidak akan memberikan dampak signifikan terhadap bagian lainnya. Implementasi prinsip ini dilakukan melalui penggunaan service layer, pemisahan logika bisnis, serta pembatasan akses langsung antar modul. Hal ini menjadikan sistem lebih fleksibel dan lebih mudah dikembangkan di masa depan.

Prinsip berikutnya adalah pendekatan berbasis domain atau *Domain-Oriented Design*, yang menyesuaikan struktur sistem dengan kebutuhan bisnis di dunia nyata. Dalam MediTrack, setiap modul dirancang berdasarkan domain utama dalam layanan kesehatan, seperti rekam medis yang dikelola dalam modul EHR, pengelolaan obat dalam modul Pharmacy, serta penjadwalan dalam modul Appointment. Pendekatan ini membuat sistem lebih mudah dipahami karena struktur kode mencerminkan proses bisnis yang terjadi.

Selain itu, aspek keamanan juga menjadi perhatian utama melalui penerapan prinsip *Security by Design*. Mengingat MediTrack menangani data sensitif seperti rekam medis pasien, sistem dirancang dengan mekanisme keamanan sejak awal, termasuk autentikasi pengguna, otorisasi

berbasis peran, enkripsi data, serta pencatatan aktivitas pengguna. Dengan demikian, sistem tidak hanya berfungsi dengan baik tetapi juga mampu menjaga kerahasiaan dan integritas data.

b. Identify trade-offs and limitations

Meskipun arsitektur Modular Monolith memiliki berbagai keunggulan, terdapat beberapa trade-off dan keterbatasan yang perlu diperhatikan. Salah satu keterbatasan utama adalah dalam hal skalabilitas, di mana seluruh aplikasi harus diskalakan secara bersamaan, meskipun hanya sebagian fitur yang mengalami peningkatan beban. Hal ini dapat menyebabkan penggunaan sumber daya yang kurang efisien dibandingkan dengan arsitektur microservices yang memungkinkan skalabilitas per layanan.

Selain itu, proses deployment pada arsitektur monolith juga memiliki risiko yang lebih tinggi. Setiap perubahan, sekecil apa pun, mengharuskan deployment ulang seluruh aplikasi, sehingga meningkatkan potensi terjadinya gangguan sistem apabila terjadi kesalahan. Hal ini menuntut adanya pengujian yang lebih ketat sebelum sistem dirilis.

Seiring dengan pertumbuhan sistem, ukuran codebase juga akan semakin besar dan kompleks. Jika tidak dikelola dengan baik, hal ini dapat menyulitkan developer dalam memahami struktur sistem serta meningkatkan risiko kesalahan dalam pengembangan. Kondisi ini juga dapat menjadi tantangan ketika jumlah tim pengembang bertambah, karena banyaknya kontribusi dalam satu codebase dapat menyebabkan konflik dan menurunkan produktivitas.

c. Suggest how the architecture supports scalability, maintainability, and extensibility

Meskipun memiliki keterbatasan, arsitektur Modular Monolith tetap mampu mendukung kebutuhan skalabilitas, maintainability, dan extensibility jika dirancang dengan tepat. Dari sisi skalabilitas, sistem dapat ditingkatkan performanya melalui pendekatan horizontal scaling dengan menambahkan server dan menggunakan load balancer untuk mendistribusikan beban. Selain itu, penggunaan caching seperti Redis dapat membantu mempercepat akses data yang sering digunakan, sementara mekanisme queue memungkinkan pemrosesan tugas secara asynchronous, seperti pengiriman notifikasi dan pemrosesan pembayaran.

Dari sisi maintainability, struktur modular yang diterapkan memungkinkan sistem lebih mudah dipelihara karena setiap bagian memiliki batasan yang jelas. Penggunaan pola desain seperti service layer dan repository pattern juga membantu menjaga kebersihan kode serta mempermudah proses debugging dan pengembangan lanjutan. Dengan struktur yang terorganisir, developer dapat dengan mudah memahami dan memperbaiki bagian tertentu tanpa

harus memahami keseluruhan sistem.

Sementara itu, dalam hal extensibility, arsitektur ini memberikan fleksibilitas untuk menambahkan fitur baru tanpa harus mengubah struktur utama sistem secara signifikan. Fitur seperti telemedicine, integrasi kecerdasan buatan, maupun pengembangan aplikasi mobile dapat ditambahkan sebagai modul baru. Selain itu, pendekatan modular juga memungkinkan sistem untuk berevolusi ke arah arsitektur microservices di masa depan dengan cara memisahkan modul tertentu menjadi layanan independen sesuai kebutuhan.

2.1.2. System Decomposition & Modeling

a. <u>List the modules/services and their responsibilities.</u>

Dalam arsitektur Modular Monolith yang digunakan pada MediTrack, sistem dipecah menjadi beberapa modul utama berdasarkan domain bisnis. Pendekatan ini bertujuan untuk menjaga keteraturan sistem, meminimalkan kompleksitas, serta mempermudah pengembangan dan pemeliharaan. Modul pertama adalah User Management, yang bertanggung jawab dalam mengelola seluruh data pengguna seperti pasien, dokter, apoteker, dan administrator. Modul ini juga menangani autentikasi, otorisasi, serta pengelolaan peran pengguna dalam sistem.

Modul kedua adalah Appointment, yang berfungsi untuk mengelola proses penjadwalan konsultasi antara pasien dan dokter. Modul ini mencakup fitur pemesanan jadwal, perubahan jadwal, serta pembatalan janji temu. Selanjutnya adalah modul Electronic Health Records (EHR) yang berperan dalam menyimpan dan mengelola data medis pasien, termasuk riwayat penyakit, hasil pemeriksaan, resep obat, serta hasil laboratorium. Modul ini menjadi salah satu komponen paling krusial karena menangani data sensitif.

Modul Pharmacy bertanggung jawab dalam pengelolaan obat, termasuk stok, distribusi, serta pemrosesan resep yang diberikan oleh dokter. Modul ini juga terintegrasi dengan EHR untuk menerima data resep secara langsung. Modul Payment digunakan untuk menangani seluruh transaksi keuangan dalam sistem, termasuk pembayaran online dan klaim asuransi. Modul ini harus mampu memastikan keamanan dan keakuratan transaksi.

Terakhir, modul Analytics digunakan untuk mengolah data yang ada dalam sistem menjadi informasi yang berguna, seperti analisis performa dokter, tren penggunaan obat, serta hasil perawatan pasien.

b. <u>For each module, define key classes/entities and their relationships.</u>

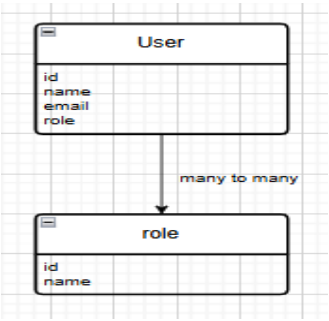
Setiap modul dalam 004DediTrack memiliki entitas utama yang merepresentasikan objek dalam sistem serta relasi antar entitas tersebut. Pada modul User Management, entitas utama terdiri dari User, Role, dan Profile. Setiap user memiliki satu role, dan setiap user memiliki satu profil yang berisi informasi detail sesuai jenis pengguna. Pada modul Appointment, terdapat entitas Appointment yang berelasi dengan Patient dan Doctor. Satu pasien dapat memiliki banyak appointment, dan satu dokter juga dapat menangani banyak appointment.

Pada modul EHR, entitas utama meliputi MedicalRecord, Prescription, dan LabResult. Setiap medical record dimiliki oleh satu pasien dan dapat memiliki banyak prescription serta hasil lab. Pada modul Pharmacy, terdapat entitas Drug, Inventory, dan Order. Prescription dari modul EHR akan menghasilkan order yang diproses oleh pharmacy. Inventory digunakan untuk mengelola stok obat. Pada modul Payment, entitas seperti Transaction dan InsuranceClaim digunakan untuk mencatat pembayaran yang dilakukan pasien serta klaim yang diajukan ke pihak asuransi. Pada modul Analytics, entitas seperti Report dan Metrics digunakan untuk mengolah dan menyajikan data dari berbagai modul lainnya. Relasi antar modul tetap dijaga seminimal mungkin dengan komunikasi melalui service layer, sehingga tidak terjadi ketergantungan langsung yang berlebihan.

c. <u>Use UML Class Diagrams or component diagrams to illustrate the design.</u>

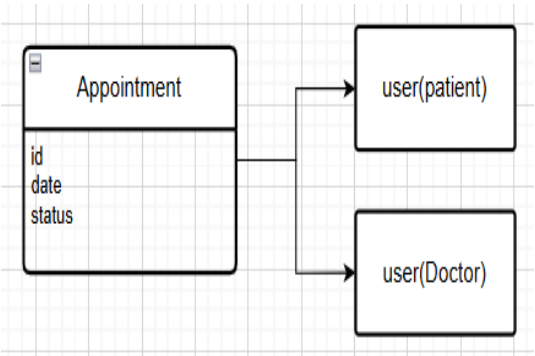
Berikut berapa class diagram yang ada dalam sistem :

- User — Role (Many-to-One)



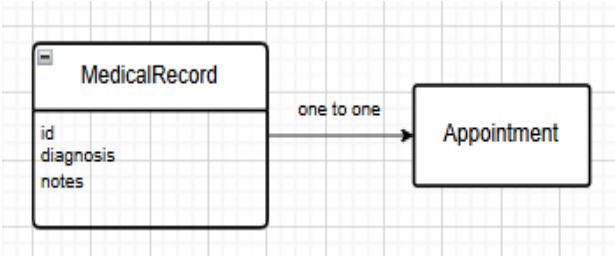
Relasi antara *User* dan *Role* adalah many-to-one, yang berarti banyak pengguna dapat memiliki satu peran yang sama. Dalam sistem MediTrack, setiap user seperti pasien, dokter, atau admin akan memiliki satu role yang menentukan hak aksesnya.

- User — Appointment (Patient & Doctor) (One-to-Many)



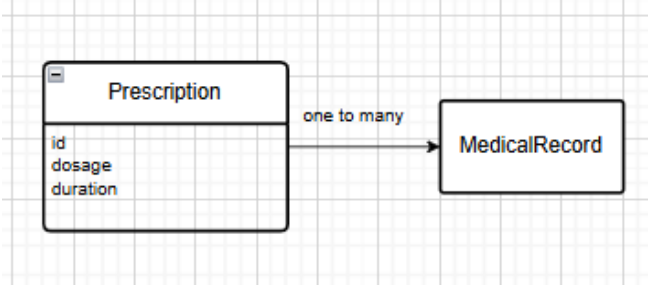
Relasi antara *User* dan *Appointment* terjadi dalam dua konteks, yaitu sebagai pasien dan sebagai dokter. Satu user yang berperan sebagai pasien dapat memiliki banyak appointment, karena pasien dapat melakukan beberapa konsultasi. Demikian pula, satu user sebagai dokter dapat menangani banyak appointment dari berbagai pasien. Setiap appointment hanya terkait dengan satu pasien dan satu dokter.

- Appointment — MedicalRecord (One-to-One)



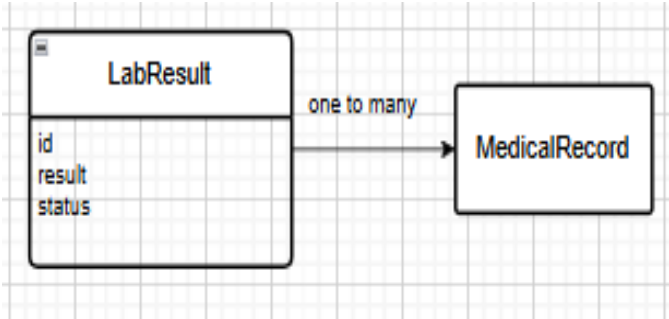
Relasi antara Appointment dan MedicalRecord adalah one-to-one, yang berarti setiap appointment menghasilkan satu rekam medis. Hal ini karena setiap sesi konsultasi akan menghasilkan satu catatan medis yang mencatat diagnosis, tindakan, dan catatan dokter.

- MedicalRecord — Prescription (One-to-Many)



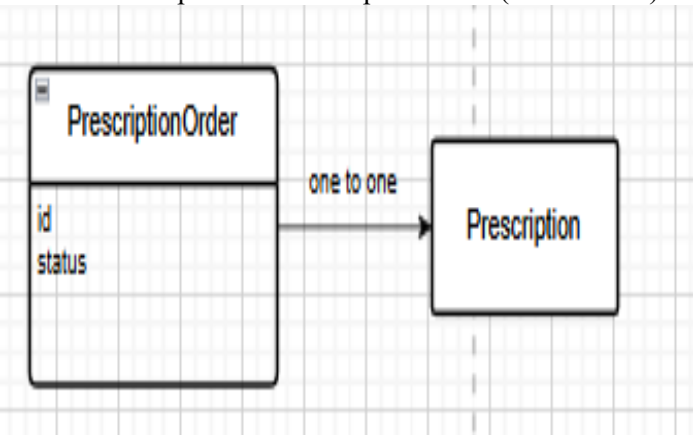
Relasi ini menunjukkan bahwa satu MedicalRecord dapat memiliki banyak Prescription. Dalam satu kunjungan medis, dokter dapat memberikan lebih dari satu resep obat kepada pasien.

- MedicalRecord — LabResult (One-to-Many)



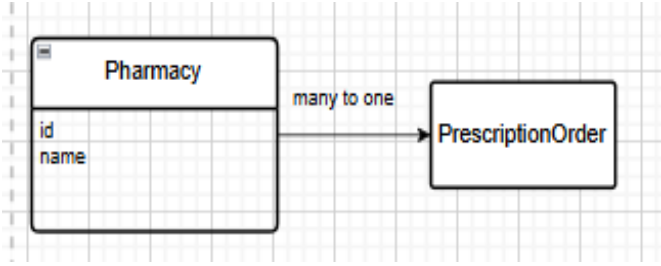
Relasi antara MedicalRecord dan LabResult bersifat one-to-many, di mana satu rekam medis dapat memiliki beberapa hasil laboratorium. Hal ini mencerminkan kondisi nyata bahwa dalam satu pemeriksaan, pasien dapat menjalani beberapa tes laboratorium.

- Prescription — PrescriptionOrder (One-to-One)



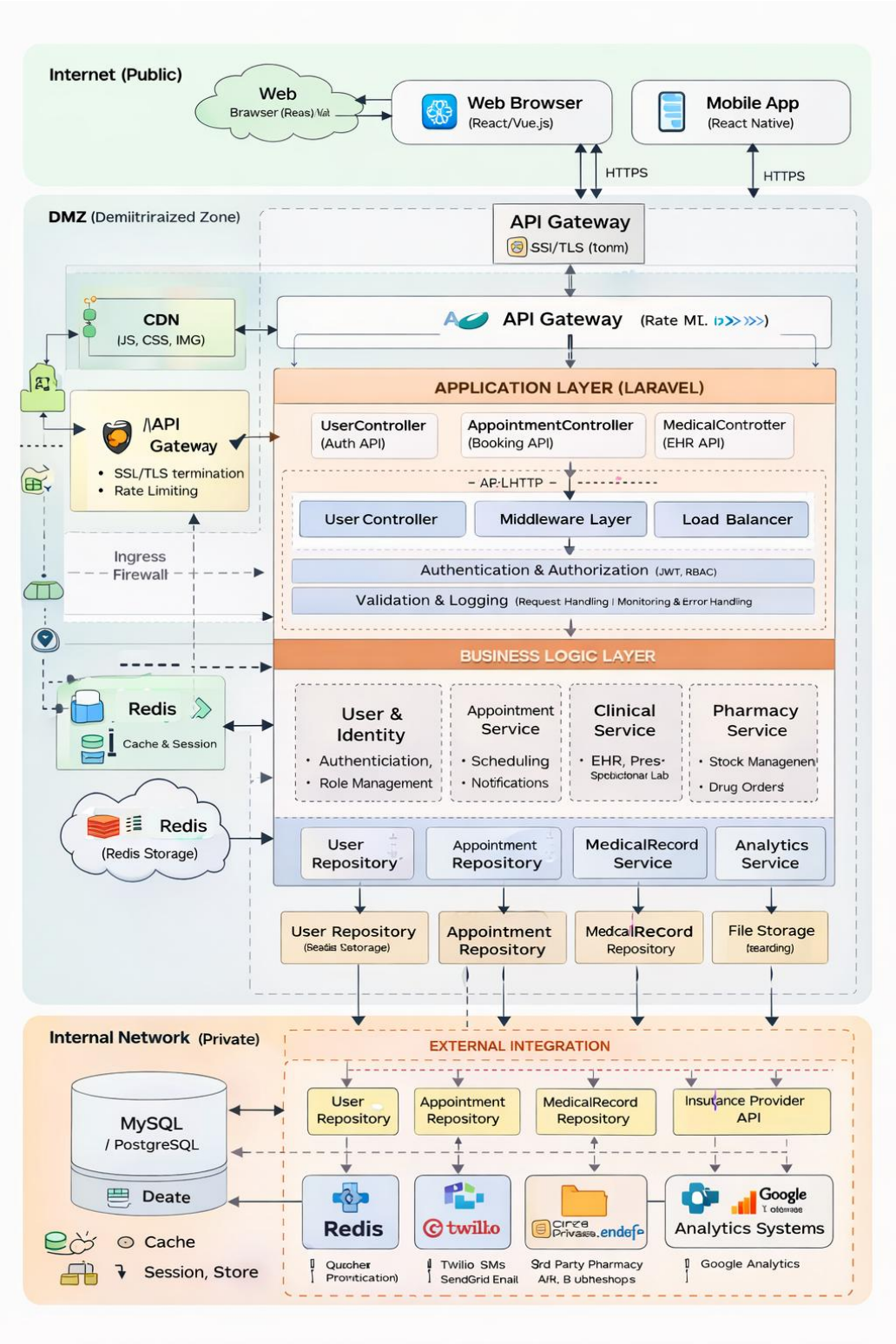
Relasi ini menunjukkan bahwa setiap Prescription akan menghasilkan satu PrescriptionOrder dalam sistem farmasi. Prescription merupakan instruksi medis dari dokter, sedangkan prescription order adalah bentuk implementasinya di apotek. Satu prescription tidak dapat memiliki lebih dari satu order utama dalam konteks sederhana ini.

- PrescriptionOrder — Pharmacy (Many-to-One)



Relasi antara PrescriptionOrder dan Pharmacy adalah many-to-one. Banyak order resep dapat diproses oleh satu pharmacy. Namun, setiap order hanya ditangani oleh satu pharmacy tertentu. Hal ini mencerminkan proses distribusi obat di dunia nyata.

2.1.3. Architecture Visualization



a. Include components, services, data flow, and integration points.

Arsitektur MediTrack terdiri dari beberapa komponen utama yang terorganisir dalam beberapa lapisan. Pada lapisan teratas terdapat **Client Layer** yang mencakup web application dan mobile application. Client berinteraksi dengan sistem melalui API menggunakan protokol HTTPS.

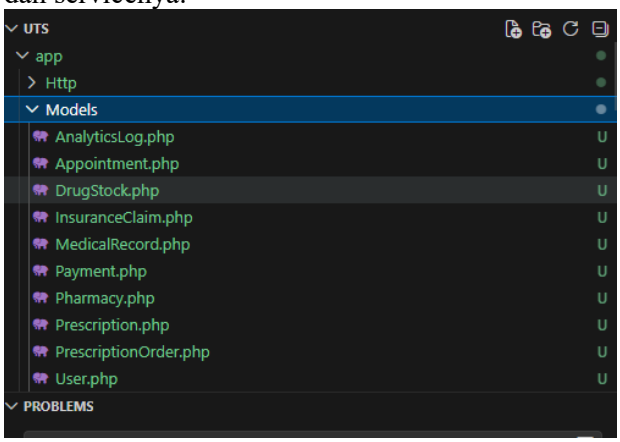
b. Clearly label boundaries

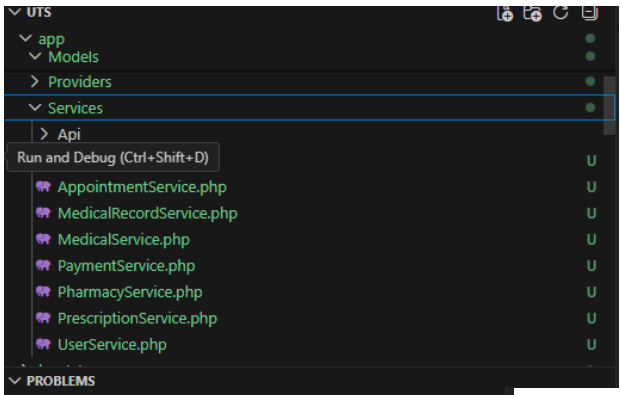
c. Indicate how external systems

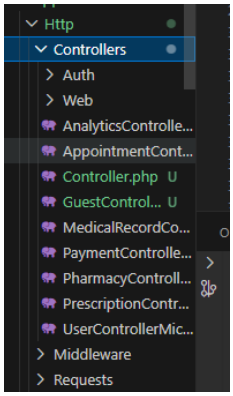
tidak ada terkait dengan eksternal sistem lain

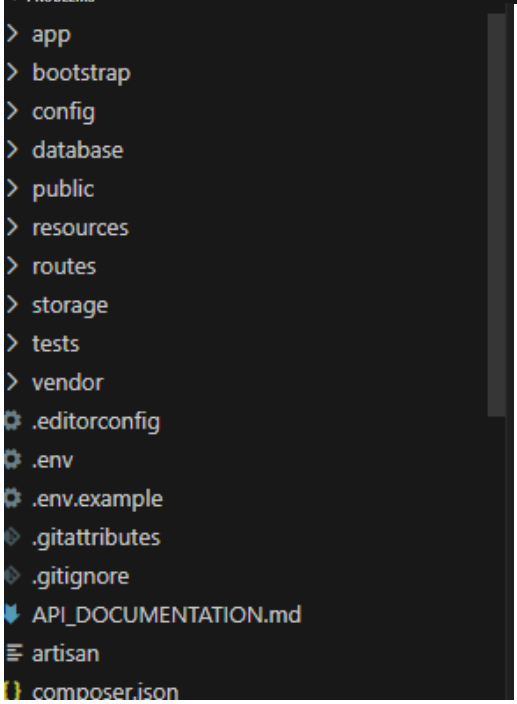
2.1.4. Implementasi (Source Code Overview)

Saya memakai arsitektur monolit
bukti:
Semua logika bisnis disatukan dalam satu proyek laravel, mulai dari database, tampilan , controller dan servicenya.

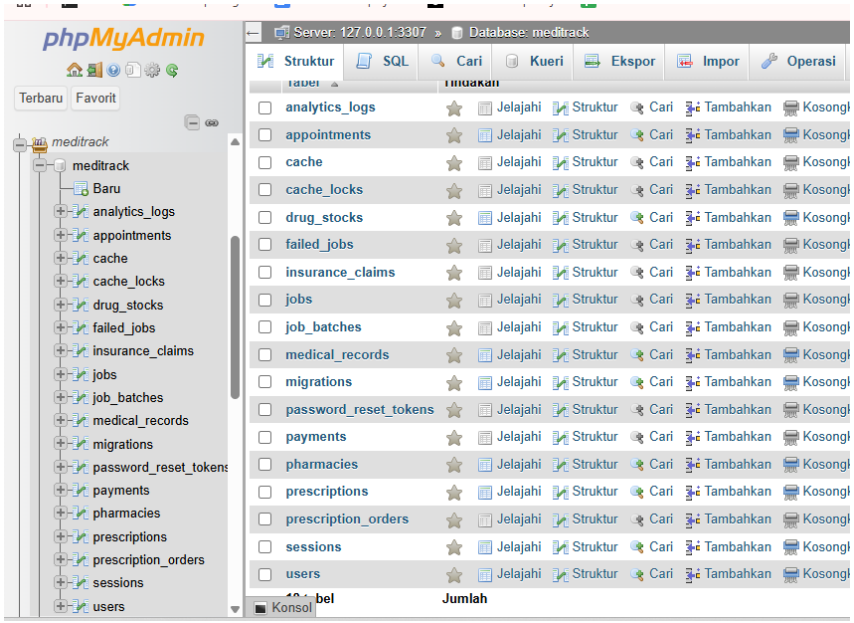








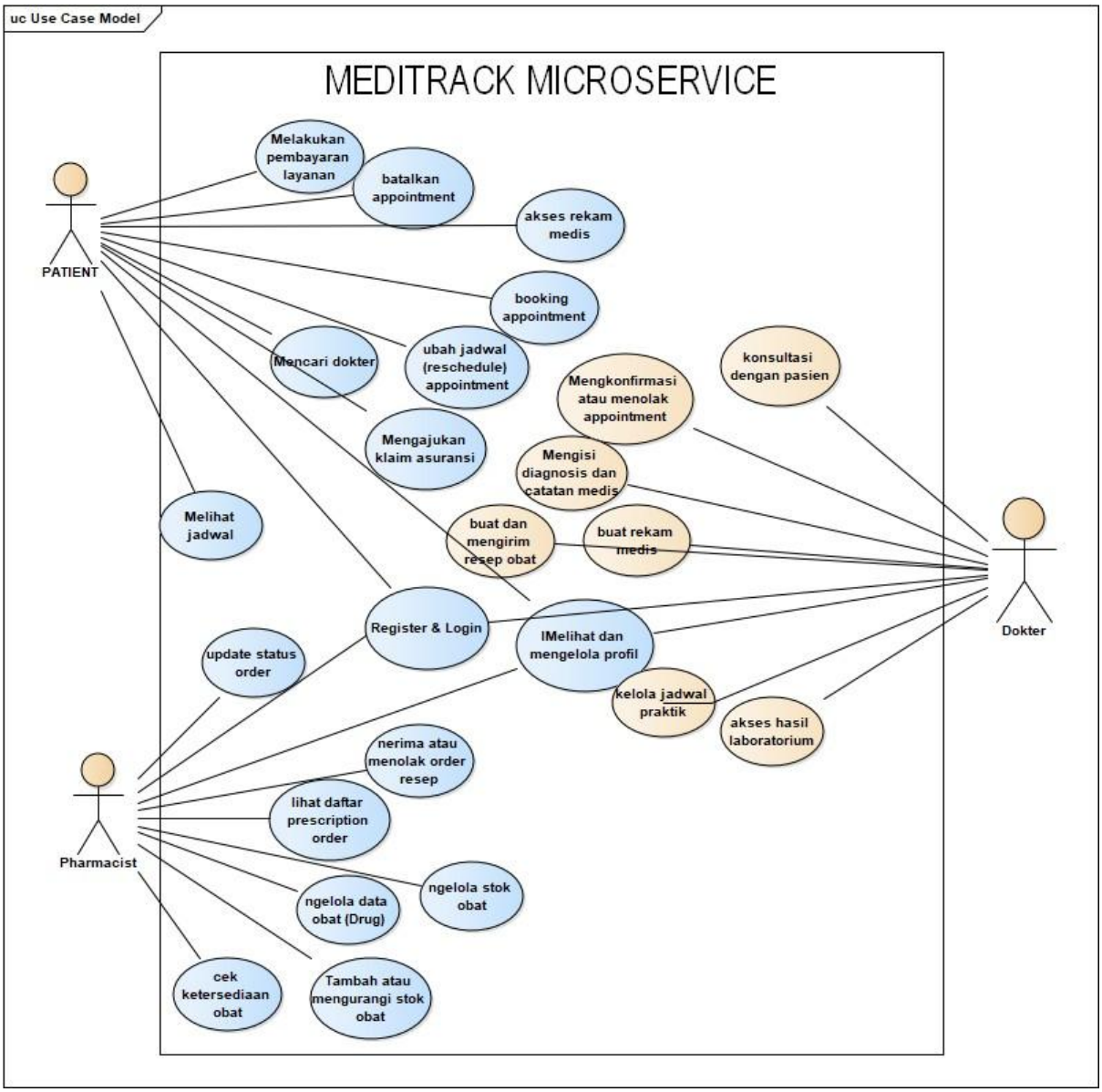
Database 1 untuk semua fitur



2.1.5. Bonus Task: Performance Bottleneck & Mitigation

2.2 Case Study 2: MediTrack Transformation

2.2.1. Use Case Based On The Case Study



1. **Patient (Pasien)**

Pasien merupakan pengguna utama yang menggunakan platform untuk mendapatkan layanan kesehatan secara digital.

- Registrasi akun sebagai pasien
- Login ke dalam sistem
- Melihat dan mengelola profil pribadi
- Mencari dokter berdasarkan spesialisasi atau lokasi
- Melihat jadwal ketersediaan dokter
- Melakukan booking appointment
- Mengubah jadwal (reschedule) appointment
- Membatalkan appointment

- Menerima notifikasi (appointment, resep, pembayaran)
- Melihat riwayat appointment
- Mengakses rekam medis (EHR)
- Melihat hasil laboratorium
- Melihat dan menebus resep obat
- Memilih pharmacy untuk pengambilan obat
- Melakukan pembayaran layanan
- Mengajukan klaim asuransi
- Melihat status pembayaran dan klaim
- Mengunduh invoice atau bukti pembayaran

2. Doctor (Dokter)

Dokter berperan dalam memberikan layanan medis dan mengelola data klinis pasien.

Daftar Use Case:

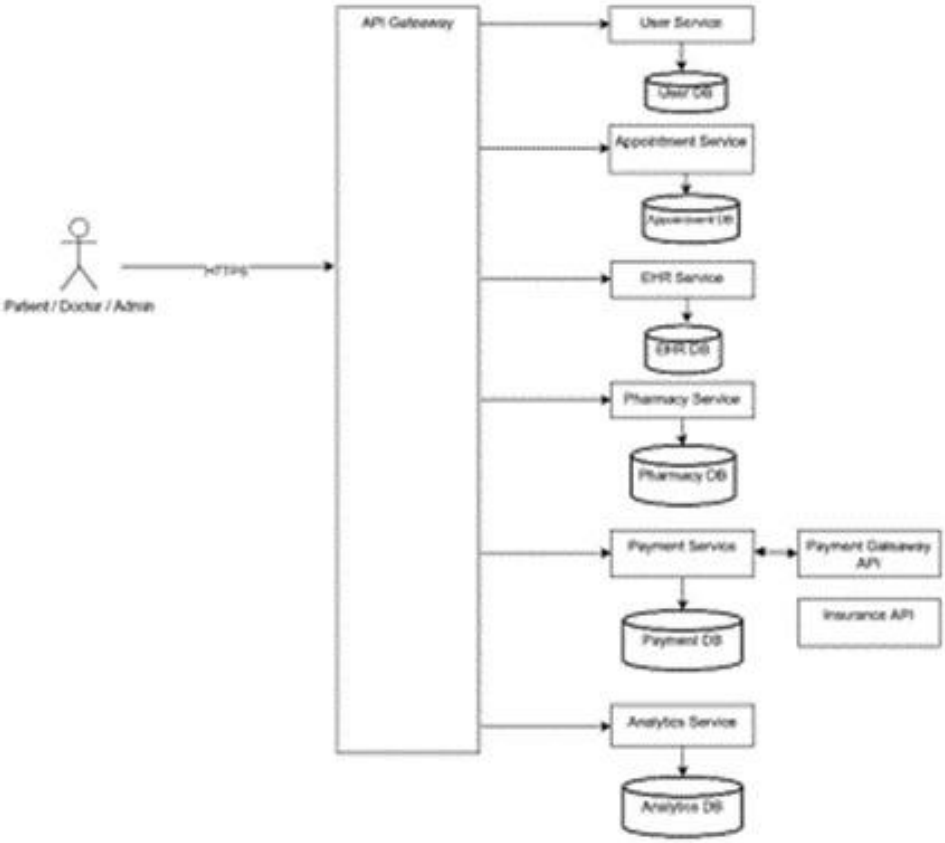
- Login ke sistem sebagai dokter
- Mengelola profil dokter (spesialisasi, jadwal, dll)
- Melihat daftar appointment pasien
- Mengkonfirmasi atau menolak appointment
- Melakukan konsultasi dengan pasien
- Membuat rekam medis (Medical Record)
- Mengisi diagnosis dan catatan medis
- Menginput hasil pemeriksaan
- Membuat dan mengirim resep obat (Prescription)
- Melihat riwayat medis pasien
- Mengakses hasil laboratorium pasien
- Mengelola jadwal praktik
- Melihat performa (analytics)
- Menerima notifikasi terkait appointment dan pasien

3. Pharmacist (Apoteker)

Apoteker bertanggung jawab dalam pengelolaan obat dan pemrosesan resep.

- Login ke sistem sebagai apoteker
- Mengelola profil pharmacy
- Melihat daftar prescription order
- Menerima atau menolak order resep
- Menyiapkan dan mendistribusikan obat
- Mengupdate status order (diproses, selesai)
- Mengelola stok obat (inventory)
- Menambah atau mengurangi stok obat
- Mengecek ketersediaan obat
- Melihat riwayat transaksi obat
- Mengelola data obat (Drug)
- Melihat laporan penggunaan obat
- Menerima notifikasi order baru

2.2.2. Architecture Diagram



a. Draw both the monolith and the microservices architecture.
b. Highlight boundaries, data flow, and integration points.
c. Show how the new architecture covers the old functionality without deleting it

2.2.3. Reasoning and Trade-offs

a. Explain the reasoning behind the transformation.
b. Discuss trade-offs (e.g., deployment complexity vs scalability).

2.2.4. Migration Strategy

2.2.5. Implementasi (Monolith Representastion)

a. Provide a simplified java code structure (classes, packages, modules) representing the monolithic architecture.
b. Show how modules are tightly coupled (e.g., shared database, direct method calls).

2.2.6. Microservices Mapping

a. Transform the monolith into microservices.
b. Map each monolith module into a microservice (Auth, Appointment, EHR, Pharmacy, Analytics, Payment).
c. Show how communication changes (REST APIs, message queues, service discovery).